

CSCE 625 ARTIFICIAL INTELLIGENCE · MODULE 11 · LECTURE NOTES

Game AI Architecture

What actually runs, sixty times a second.

MODULE OUTCOMES

Compare finite state machines, behaviour trees, and utility systems.

Implement a behaviour tree and explain its composability.

Implement steering behaviours and local avoidance.

Design for a frame budget across many agents.

Choose an architecture and justify it against the alternatives.

1 The frame budget

THE FRAME BUDGET IS THE DESIGN CONSTRAINT, WITH ARITHMETIC

Sixty frames per second is 16.7 milliseconds total, shared between rendering, physics, animation, audio, networking, game logic, and every agent's decision-making.

AI typically gets one to two milliseconds of that. With fifty active agents that is twenty to forty microseconds per agent per frame — which is a few thousand instructions, and will not accommodate a search.

So most agents must do nothing on most frames. The standard answers are **staggered updates** (each agent re-decides every n-th frame, with the phases spread so the cost is even), **level-of-detail AI** (distant or off-screen agents think less often and more coarsely, or are simulated statistically), and **event-driven rather than polled logic** (react when something happens rather than checking every frame whether it has).

And the architecture must be interruptible and incremental — which rules out anything that must run to completion before producing a usable answer, and is exactly why anytime algorithms keep appearing in this course (Module 03 §4's anytime A*, Module 05 §3's MCTS, Module 04 §3's time-slicing). **The frame budget is not a detail of game AI; it is the reason game AI selected the methods it did.**

	Finite state machine	Behaviour tree	Utility system
Decides by	Current state plus transition conditions.	Walking a tree from the root each tick.	Scoring every available option.
Authoring cost	$O(n^2)$ transitions — every state potentially needs an edge to every other.	Composable subtrees; linear in behaviours.	Tune a curve per consideration per action.

	Finite state machine	Behaviour tree	Utility system
Scales to	About ten states before it is unmanageable.	Hundreds of nodes.	Many options, with care.
Debugging	Easy — you are in one state.	Easy — inspect the currently active path.	Hard — 'why did that score highest?'
Best for	Simple agents with few genuine modes.	The default choice for most games.	Agents balancing many competing concerns.

Behaviour trees won because of the authoring column. The transition explosion is what killed state machines — adding a 'stunned' state to a ten-state machine means considering ten new transitions in and ten out, and the designer must get all twenty right — **and composability is what fixed it.**

2 Behaviour trees

Every node, when ticked, returns SUCCESS, FAILURE, or RUNNING. That is the entire interface.

```

SEQUENCE    tick children in order; fail immediately if any
             child fails; succeed when all have succeeded.
             ("AND", or "do this, then this")
SELECTOR    tick children in order; succeed immediately if
             any child succeeds; fail if all fail.
             ("OR", or "try this, otherwise this")
DECORATOR    wrap exactly one child and modify it: invert
             its result, repeat it, add a cooldown, add a
             precondition guard, limit its runtime.
LEAF         an action to perform, or a condition to check.

```

THAT IS IT. Four node types.

WHY IT COMPOSES: a subtree's interface is identical to a leaf's, so ANY subtree can be substituted anywhere a node is expected. Build "FightEnemy" once, reuse it in twelve places, and improve it once for all twelve.

-- which is why the authoring cost is LINEAR in the number of behaviours rather than QUADRATIC in the number of states.

RUNNING is the part people get wrong. It means "I am not finished; tick me again next frame". It is what makes the tree usable in a game loop at all -- and it means the tree carries state between ticks (which node is currently running), so A BEHAVIOUR TREE IS NOT A PURE FUNCTION OF THE WORLD. Implementations that forget this restart long actions every frame.

PRIORITY IS STRUCTURAL: a selector's left-to-right child order IS the priority order. Put "flee if nearly dead" leftmost, and it pre-empts everything to its right.

The substitutability of subtrees is the whole advantage, and it is the same compositional property that makes a good API good — a uniform interface at every level of granularity, so that composition does not require knowing what you are composing. **Which is why behaviour trees are authorable by designers rather than only by programmers**, and that, rather than any runtime property, is why they are the industry default.

UTILITY SYSTEMS, AND WHEN THEY BEAT TREES

Score every available action with a utility function over the current world state, then take the highest-scoring one. No structure at all — a flat list of options and a scoring function per option.

The strength is handling many competing considerations. Health, ammunition, distance to target, availability of cover, nearby allies, cooldown states — **each contributes to each action's score through a tunable response curve**, and the combination handles situations nobody enumerated. **A behaviour tree encoding the same trade-offs needs a condition per combination, which is the transition explosion in another form.**

And it degrades gracefully. There is always a highest-scoring action, so there is no 'no valid behaviour' state — which a behaviour tree does reach (every child of the root selector fails) and must handle explicitly with a fallback.

The weakness is debuggability. 'Why did it do that?' becomes a matter of comparing twenty scores and the dozen curve evaluations behind each, and tuning one curve shifts every decision that curve participates in — so changes are non-local in a way behaviour-tree edits are not. **Log every score, always**, and ship the log in a debug overlay: it is the only thing that makes the architecture maintainable.

3 Movement and steering

- **Seek, flee, and arrive.** A desired velocity toward or away from a target point, with *arrive* scaling the desired speed down as the distance falls so the agent stops rather than overshooting and oscillating.
- **Path following.** Seek the next point on the path from Module 04, **looking ahead along the path rather than at the immediate next node**, which makes the agent cut corners naturally and removes the need for much of §2's smoothing.
- **Separation, alignment, and cohesion. The three rules that produce flocking** — stay apart, match neighbours' headings, move toward the local centre — and **the classic demonstration that simple reflex agents (Module 01 §1) produce complex collective behaviour** with no coordination, no plan, and no communication.
- **Obstacle avoidance and reciprocal velocity obstacles. RVO has each agent assume the others are also avoiding, and take half the required correction** — which prevents the oscillation that naive mutual avoidance produces, where both agents dodge the same way, then both dodge back.
- **Composed by weighted sum, or by priority. Weighted sums cancel:** two avoidance forces from opposite walls in a corridor sum to approximately zero, **and the agent calmly walks into the wall ahead** because the only surviving component was the seek. **This is the classic steering bug. Prioritised composition (take the highest-priority non-zero behaviour) or truncated accumulation (add behaviours until the force budget is used) avoids it**, at the cost of less smooth motion — which is the right trade, since smoothness is cosmetic and walking into walls is not.

4 Choosing, and legibility

LAYER BY TIMESCALE

Strategic layer (seconds): which goal to pursue. A behaviour tree, a utility score, or a planner (Module 07). **Runs rarely** — every few hundred milliseconds, or on events.

Tactical layer (hundreds of milliseconds): which path, which cover point, which target. Pathfinding (Module 04) and spatial queries. **Runs on demand**, and is time-sliced when it must.

Steering layer (every frame): where to move right now. Reflex behaviours only (§3) — Module 01 §1's cheapest agent design, doing the most frequent work, which is the correct assignment and not a compromise.

Each layer runs at its own rate, and the division is by how fast the relevant information changes. Which is the same decomposition as Module 04 §4's three movement layers, and getting the division wrong — putting avoidance in the planner, or target selection in the steering — is the commonest architectural mistake in game AI, because it couples a fast-changing input to a slow-running computation.

- **Telegraph intent.** An animation, a sound, or a stance change before the action, so the player has time to respond. **A perfectly-timed instant attack reads as unfair rather than as skilled**, and **deliberately**

slowing an agent's reaction is a feature — which is the clearest case of optimality being the wrong objective (Module 05 §4).

- **Commit visibly. An agent that changes its mind every frame appears broken even when every individual decision is correct**, because the player sees indecision rather than responsiveness. **Add hysteresis** — require a new option to beat the current one by a margin — which is Module 04 §4's oscillation fix at a higher level.
- **Fail legibly.** If the agent cannot reach you, it should visibly try and fail — approach, pause, look, reposition — **rather than stand still**, which reads as a bug even when it is the correct response to an unreachable target.
- **And log the decision. Ship a debug overlay showing each agent's active behaviour and the reason for it** — the active tree path, or the top three utility scores. **It pays for itself within a week** and it is the practical form of Module 01 §4's debuggability argument.
- **Legibility is a functional requirement rather than polish. A player who cannot predict an agent cannot play against it, so an unreadable agent is a broken agent however good its decisions are** — which means legibility belongs in the architecture's requirements alongside the frame budget, and not in a list of nice-to-haves.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Game AI Pro, volumes 1–3 (free online)	The reference for this entire module — chapters by shipping practitioners on behaviour trees, utility systems, steering, and the frame-budget engineering of §1.
Millington — AI for Games, 3rd edition	The systematic treatment of §2 and §3 , with pseudocode throughout. Library copy.
Colledanchise & Ögren — Behavior Trees in Robotics and AI (free PDF)	§2 treated formally , including the relationship to state machines and the composability argument made precise.
Reynolds — Steering Behaviors For Autonomous Characters (free)	The §3 behaviours, from the originator , with demonstrations. The flocking paper is here too.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Compute your own per-agent frame budget** for a target agent count and frame rate.

- 2 **Implement staggered updates** and measure the frame-time variance before and after.
- 3 **Build a ten-state FSM** for an agent, then add one state and count the transitions you had to consider.
- 4 **Rebuild the same behaviour as a behaviour tree** and compare the authoring effort.
- 5 **Implement the four node types** and confirm a subtree can substitute for a leaf.
- 6 **Implement RUNNING incorrectly** (restart every frame) and observe the symptom.
- 7 **Build a utility system** for the same agent and log every score each tick.
- 8 **Change one curve** and report how many decisions changed.
- 9 **Implement flocking** with the three rules and 200 agents.
- 10 **Reproduce the weighted-sum cancellation bug** in a corridor, then fix it by prioritised composition.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Compute the per-agent frame budget and name three ways to live within it.
- 2 Why must a game AI architecture be interruptible?
- 3 Compare FSMs, behaviour trees, and utility systems on five axes.
- 4 Why did behaviour trees displace state machines?
- 5 Name the four node types and state why subtrees compose.
- 6 What does RUNNING mean, and what does it imply about purity?
- 7 When does a utility system beat a tree, and what does it cost?
- 8 Name five steering behaviours and the cancellation bug.
- 9 Give the three layers and the principle dividing them.
- 10 Why is legibility a functional requirement?